

Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: uoh-sqak

Course 203.3763 — Orchestration of AI Agents

University of Haifa, Spring 2026 (תשפ"ו)

Lecturer: Dr. Yoram Reuven Segal

June 10, 2026

Generated by a CrewAI multi-agent pipeline

GitHub: <https://github.com/salah-dev-stu/uoh-sqak-ex03>

Contents

1	Introduction to Multi-Agent Orchestration	3
1.1	The Problem with Single-Agent Systems	3
1.2	Key Roles in Multi-Agent Orchestration	4
1.3	Orchestration vs. Linear Pipelines	4
1.4	Article Roadmap	5
2	Agent Architectures and Orchestration Topologies	6
2.1	Agent Perception and Autonomy	6
2.2	Linear Sequential Orchestration	6
2.3	Hierarchical Manager-Worker Orchestration	7
2.4	Peer-to-Peer Consensus Orchestration	7
2.5	Tool-Augmented Specialist Orchestration	7
2.6	Topology Comparison	8
2.7	Task Dependencies and Context Passing	8
2.8	Tool Access and Capabilities	9
2.9	Reliability Patterns	9
3	Orchestration Frameworks	10
3.1	LangChain and LCEL	10
3.2	CrewAI Role-Based Agents	10
3.3	LangGraph Stateful Graphs	11
3.4	Microsoft AutoGen	11
3.5	Framework Comparison	11
3.6	Trade-Offs and Selection Guidance	12
3.6.1	CrewAI for Sequential, Role-Based Workflows	13
3.6.2	LangGraph for Complex, Conditional Workflows	13
3.6.3	AutoGen for Iterative Refinement	13
3.6.4	LangChain LCEL for Custom Orchestration	13
4	Production Patterns and Challenges	14
4.1	Rate Limiting and Token Budgets	14
4.2	Gatekeeper Pattern	14
4.3	Retry and Circuit Breaker	15
4.4	Observability and Logging	15
4.5	Cost Optimization	17
18	5 דו-כיוונית: עברית ואנגלית במערכות סוכנים	5
18	משמעות הדו-כיוונית	5.1
18	פתרון LuaLaTeX ו-Polyglossia	5.2

19	אתגרי ייצור בסביבה דו-כיוונית	5.3
5.4	Mixed-Language Example	19
6	Case Study: This Article’s Production Pipeline	21
6.1	Overview	21
6.2	The Crew: Four Specialized Agents	21
6.3	Workflow Architecture	22
6.4	Key Engineering Decisions	23
6.5	Pipeline Metrics	24
7	Conclusion	25
7.1	Lessons Learned	25
7.2	Future Directions	26
	Bibliography	27

Chapter 1

Introduction to Multi-Agent Orchestration

Multi-agent orchestration represents a fundamental shift in how we design artificial intelligence systems. Rather than deploying a single monolithic language model, modern applications coordinate multiple specialized agents—each with distinct roles, capabilities, and knowledge domains—toward shared objectives [3]. This distributed approach mirrors human team dynamics: a researcher gathers information, an analyst interprets findings, and a writer synthesizes results into coherent narratives.

The motivation is compelling. Single-agent systems struggle with complex workflows requiring diverse skills. A researcher agent may excel at information gathering but produce verbose, unstructured output unsuitable for publication. By composing specialized agents, we achieve both specialization and division of labor—the researcher focuses on accuracy and comprehensiveness, while the writer optimizes for clarity and engagement [5]. This orchestrated approach transforms rigid pipelines into adaptive, self-correcting processes.

1.1 The Problem with Single-Agent Systems

Monolithic language models face inherent limitations when tasked with complex, multi-stage workflows. A single LLM attempting to research, analyze, and write simultaneously must juggle incompatible objectives: accuracy (favoring verbose citations and hedging language) conflicts with readability (favoring conciseness and narrative flow). Cognitive load increases, token consumption rises, and output quality often suffers from diluted focus.

Furthermore, single-agent systems lack specialization incentives. We cannot configure a researcher agent with “find comprehensive sources and verify credibility” while simultaneously configuring a writer agent with “produce engaging prose without citations cluttering the text.” The unified agent must compromise on both dimensions. Multi-agent orchestration solves this by allowing role-specific prompting, backstory configuration, and autonomous execution scopes [15].

Single-agent systems also struggle with error recovery. If a web search returns insufficient results, a single agent might either abandon the task or waste tokens on remedial searches. A multi-agent team can designate a fact-checking agent to validate intermediate results and signal researchers to refine queries. This adaptive capability requires explicit orchestration—the kind of coordination that multi-agent frameworks provide.

1.2 Key Roles in Multi-Agent Orchestration

Effective orchestration decomposes complex workflows into agent roles, each optimized for a specific contribution:

The Researcher Agent

Gathers information from external sources (web search, document databases, APIs). Specialization: query formulation, source credibility assessment, comprehensive coverage. The researcher produces raw findings with source citations, not final prose. Output is deliberately verbose and annotated, enabling downstream agents to verify claims and assess information quality.

The Writer Agent

Synthesizes findings into coherent, engaging narratives. Specialization: structure, clarity, narrative flow, audience adaptation. The writer consumes researcher outputs, selects the strongest claims, and crafts prose suitable for publication. The writer is encouraged to condense and reorganize, focusing on readability over exhaustive citation.

The Editor Agent

Reviews drafts for consistency, tone, factuality, and alignment with publication standards. Specialization: quality assurance, contradiction detection, stylistic coherence. The editor may flag passages for researcher follow-up, request rewrites from the writer, or coordinate with other agents to resolve discrepancies.

The LaTeX Producer Agent

Transforms final prose into publication-ready LaTeX markup, including structural elements (chapters, sections, references), formatting (bold, italics, code), mathematical notation, and bibliography integration. Specialization: LaTeX syntax, BibTeX/Biber integration, figure and table placement, cross-reference accuracy.

Each agent operates with reduced cognitive load, focused prompting, and role-specific validation criteria. Orchestration engines coordinate their efforts, routing outputs between agents and managing dependencies.

1.3 Orchestration vs. Linear Pipelines

Traditional document generation follows a linear pipeline: retrieve information >> extract key points >> write draft >> compile to PDF. Each stage is a stateless function; feedback loops are rare, and errors in early stages cascade downstream.

Orchestration introduces autonomy and adaptive control. An orchestration engine monitors agent progress, detects quality issues, and triggers corrective actions. If the researcher discovers contradictory sources, it can autonomously invoke a fact-checking sub-task. If the writer detects missing context, it can request additional research without human intervention. This feedback capability transforms linear workflows into dynamic, self-correcting systems [4].

Orchestration also enables parallel execution. While the researcher gathers information, other agents might prepare templates or validate citation formats in parallel. This parallelism reduces latency and token consumption compared to strict sequencing.

1.4 Article Roadmap

This article explores multi-agent orchestration across seven chapters, progressing from conceptual foundations to production implementation and real-world application:

- Chapter 1** introduces orchestration motivation, key agent roles, and the distinction from linear pipelines.
- Chapter 2** examines agent architectures and orchestration topologies: sequential, hierarchical, and state-machine models.
- Chapter 3** compares three production frameworks (CrewAI, LangGraph, AutoGen), analyzing their trade-offs and selection criteria.
- Chapter 4** addresses production deployment: scaling, reliability patterns, observability, cost optimization, and human-in-the-loop integration.
- Chapter 5** explores bilingual and bidirectional (BiDi) text handling, essential for Hebrew–English document generation.
- Chapter 6** presents a comprehensive case study: a multi-agent system that generates a 15-page LaTeX article on arbitrary topics, demonstrating end-to-end orchestration in practice.
- Chapter 7** synthesizes lessons learned, emerging best practices, and future directions for multi-agent orchestration research and deployment.

Multi-agent orchestration is no longer theoretical—it is production-ready and increasingly essential for building AI systems that are robust, maintainable, and scalable. The discipline of orchestration separates systems that gracefully adapt from those that fail catastrophically when single components malfunction. As language models grow more capable and enterprises demand higher-quality AI outcomes, orchestration frameworks will become as fundamental to software engineering as web frameworks and database layers.

Chapter 2

Agent Architectures and Orchestration Topologies

Agent architectures determine how agents perceive their environment, maintain internal state, and coordinate with peers. Understanding these topologies is essential for selecting appropriate frameworks and designing reliable multi-agent systems.

2.1 Agent Perception and Autonomy

An agent is an autonomous computational entity that perceives its environment through tools and knowledge bases, maintains internal state (conversation history, intermediate results), and takes actions to achieve goals [9]. Modern AI agents are powered by large language models (LLMs) with access to external tools: web search APIs, document analysis services, code execution environments, and data transformation pipelines.

This autonomy introduces both opportunity and risk. Agents can intelligently sequence operations based on runtime observations: if a researcher discovers that an initial web search returned insufficient results, it can autonomously refine its query and retry. However, this flexibility also means agents may make errors, hallucinate, or diverge from intended behavior—production systems must implement safeguards (fact-checking agents, output validation, human review gates).

2.2 Linear Sequential Orchestration

Task execution proceeds linearly: Task A completes, outputs flow to Task B, which begins immediately. The Researcher Agent completes its investigation, passing findings to the Analyst Agent, who then passes refined analysis to the Writer Agent [3].

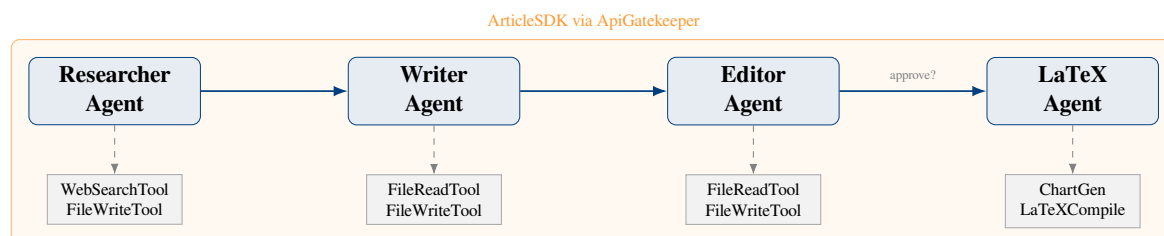


Figure 2.1: CrewAI multi-agent architecture.

Advantages: Simple to understand and debug; natural for linear workflows (research >

analysis > writing). The latency of a sequential pipeline is the sum of all individual task latencies:

$$L_{\text{seq}} = \sum_{i=1}^N \ell_i \quad (2.1)$$

Disadvantages: No parallelism; if an upstream task fails, dependent tasks cannot proceed. This blocking behavior makes sequential pipelines unsuitable for workflows where independent subtasks could execute concurrently.

2.3 Hierarchical Manager-Worker Orchestration

A supervisor agent delegates work to subordinate agents and aggregates results. The supervisor assigns research tasks to multiple researcher agents (one per market segment), collects their reports, and instructs the synthesizer agent to unify findings into a coherent analysis [3].

Advantages: Natural for delegation; enables parallel execution of independent subtasks. Worker agents operate concurrently, reducing overall wall-clock time compared to sequential execution.

Disadvantages: The supervisor becomes a bottleneck when aggregation complexity grows. Error propagation from subordinates can be complex, requiring careful validation and error handling at the supervisor level.

2.4 Peer-to-Peer Consensus Orchestration

In consensus-based topologies, agents operate as peers with no strict hierarchy. A debate topology, for instance, assigns proposing agents to advance alternative arguments, while a judge agent evaluates the reasoning quality of each proposal and renders a decision [7].

Advantages: Enables diverse viewpoints and adversarial validation; agents can challenge each other's assumptions, improving output quality. No single agent is critical; if one fails, others can continue.

Disadvantages: Requires explicit resolution mechanisms to avoid indefinite debate or contradiction. Token cost scales with agent count; a 5-agent debate consumes approximately 5x the tokens of a single-agent solution.

2.5 Tool-Augmented Specialist Orchestration

Each agent is specialized for a distinct capability (research, analysis, writing, validation) and equipped with domain-specific tools. The researcher accesses web search, databases, and document retrieval; the writer accesses typography and formatting tools; the validator accesses fact-checking services [5].

Advantages: Separation of concerns makes each agent's logic simpler and more testable. Specialist prompts are tailored to their domain, improving quality. Tools are registered once per agent, reducing configuration complexity.

Disadvantages: Tool proliferation increases system complexity. Tool access must be mediated by a Gatekeeper (rate limiting, permission checking, cost tracking).

2.6 Topology Comparison

Topology	Latency	Parallelism	Complexity	Ideal For
Linear Sequential	High (sum of all tasks)	None	Low	Simple workflows
Hierarchical	Medium (supervisor overhead)	High (workers)	Medium	Delegation, parallel subtasks
Peer-to-Peer	Variable (debate rounds)	Medium (peers)	High	Validation, consensus
Tool-Augmented	Low (tools reduce reasoning)	Medium (tool parallelism)	High (tool registry)	Specialist tasks
Hybrid (State Machine)	Low (conditional branching)	High (DAG parallelism)	Very High	Complex workflows

Each topology trades latency against parallelism, complexity, and operational cost. Sequential topologies minimize code complexity at the cost of wall-clock latency; hierarchical and DAG-based topologies enable parallelism but require more sophisticated orchestration logic. Tool-augmented specialists reduce per-agent reasoning cost (and thus token consumption) by offloading domain-specific tasks to external services.

The selection of topology should be driven by workflow characteristics: Is the workflow inherently linear, or do multiple independent tasks exist? Must agents validate each other's work, or is single-pass execution sufficient? What is the acceptable latency window, and how does it map to operational cost?

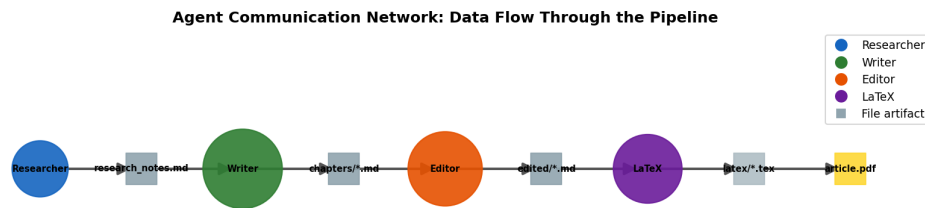


Figure 2.2: Network graph of agent communication topology in a CrewAI sequential crew. Nodes represent agents; directed edges represent task context flow.

2.7 Task Dependencies and Context Passing

Orchestration systems manage task dependencies—the requirement that Task B cannot begin until Task A completes and produces required outputs. Context passing mechanisms vary across frameworks:

- **CrewAI** [3]: Tasks implicitly pass context through the Crew object; later tasks access outputs from earlier tasks.
- **LangGraph** [4]: Explicit state dictionary that all nodes read and write to; state updates are governed by user-defined reducer functions.
- **AutoGen** [6]: Conversation history is the shared state; agents review dialogue to coordinate on prior results.

These mechanisms enable agents to build upon prior work, creating increasingly refined outputs through iterative enhancement. A researcher provides initial findings; an analyst structures those findings; a writer produces prose; an editor validates and polishes. Each stage refines the output without duplicating the upstream computation.

2.8 Tool Access and Capabilities

Tools extend agent capabilities beyond language generation. A researcher agent might access:

- Web search (retrieve current information)
- Document parsing (extract data from PDFs)
- Data retrieval (query databases)
- Code execution (run Python for calculations)

Tools are registered with agents at configuration time. The orchestration engine handles tool invocation middleware: checking permissions, enforcing rate limits, retrying on transient failures, and logging all tool invocations for audit trails. A Gatekeeper pattern (detailed in Chapter ??) centralizes this control, ensuring no tool invocation escapes cost tracking or permission checks.

2.9 Reliability Patterns

Production architectures must handle partial failures and stochastic behavior. Key patterns include:

- **Retry logic** with exponential backoff for transient failures
- **Fallback agents**: if primary researcher fails, activate secondary
- **Timeout boundaries**: agents must complete within defined windows
- **Graceful degradation**: incomplete data is preferable to failed workflow

These patterns reflect the reality that multi-agent systems are inherently probabilistic—each agent invocation incurs token cost and latency, and may fail or produce suboptimal output. A production system must explicitly account for this uncertainty and design recovery mechanisms that preserve workflow continuity even when individual components fail.

Chapter 3

Orchestration Frameworks

Three major frameworks dominate the multi-agent orchestration landscape, each embodying a distinct architectural philosophy. Understanding their trade-offs is essential for selecting the right tool for your workflow [3–5].

3.1 LangChain and LCEL

LangChain provides a foundational abstraction layer for building language model applications [5]. The Language Chain Expression Language (LCEL) is a declarative syntax for composing language models, prompts, and tools into sequential pipelines. Each component receives an input, performs transformation, and passes output to the next component. LCEL’s strength lies in its flexibility: workflows can be arbitrarily complex, with conditional branching, looping, and tool invocation woven naturally into the computation graph.

LangChain is particularly effective as a substrate layer beneath higher-level frameworks. Many CrewAI and LangGraph implementations internally use LangChain’s LLM interfaces and tool-invocation machinery. For simple sequential workflows or when building custom orchestration logic, LangChain LCEL offers fine-grained control at the cost of more boilerplate code.

3.2 CrewAI Role-Based Agents

CrewAI specializes in expressing multi-agent systems using human-intuitive abstractions [3]. Agents have roles (`Senior Researcher`), goals (`Find comprehensive market trends`), and backstories (`You are a researcher with 10 years of industry experience...`). Tasks are discrete units of work assigned to agents, with explicit dependencies and output specifications. This declarative style makes CrewAI ideal for rapid prototyping and team-oriented workflows.

A typical CrewAI workflow involves defining agents, assigning tasks with clear input/output contracts, and orchestrating them through a `Crew` object that manages execution flow. The framework handles context passing and task sequencing automatically, reducing boilerplate compared to lower-level approaches.

3.3 LangGraph Stateful Graphs

LangGraph models workflows as explicit state machines: nodes represent computation steps (agent reasoning, tool invocation, data transformation), and edges define transitions (deterministic or conditional) [4]. All agents share a centralized state dictionary, enabling natural information passing and context preservation. The state is immutable; each node receives the current state and returns a delta that LangGraph merges forward.

LangGraph excels at complex workflows requiring branching, looping, and conditional logic. Because state is explicit and all transitions are defined in code, LangGraph workflows are highly testable and debuggable. Runtime inspection of the state graph reveals exactly which computation step failed and why.

3.4 Microsoft AutoGen

AutoGen frames orchestration as a conversation problem: agents communicate through message passing, with a conversation manager coordinating turn-taking [6]. Each agent can invoke tools mid-dialogue, and tool results are incorporated into the conversation for all agents to observe. AutoGen includes sandboxed Python code execution, making it particularly effective for iterative refinement and debugging scenarios.

The conversational model is natural for workflows where agents iteratively improve an output (e.g., a code block is written, executed, debugged, and refined in multiple turns). However, the model can be verbose, increasing token consumption for workflows that do not require iteration.

3.5 Framework Comparison

Framework	Architecture	Strengths	Best For
CrewAI	Agents + Tasks (declarative)	Rapidly expressible team dynamics; minimal boilerplate; intuitive role-based specification; built-in task dependency resolution	Report generation, sequential analysis pipelines, team-oriented workflows
LangGraph	State Machine (nodes + edges)	Explicit control flow that is testable; natural support for loops, branches, and parallelism; fine-grained observation and intervention; conditional routing	Complex workflows, conditional logic, multi-stage pipelines requiring state inspection

Framework	Architecture	Strengths	Best For
LangGraph	Conversational (message passing)	Sandbox code execution out of the box; natural iteration and refinement through dialogue; agents view prior tool outputs; effective for debugging	Iterative refinement, code debugging, tool-use workflows, multi-turn conversations
LangChain	Composable pipeline (LCEL)	Flexible; fine-grained control; LCEL integrates with all frameworks; strong ecosystem of integrations	Building custom orchestration, when frameworks do not fit, substrate for higher-level systems

Continued on next page...

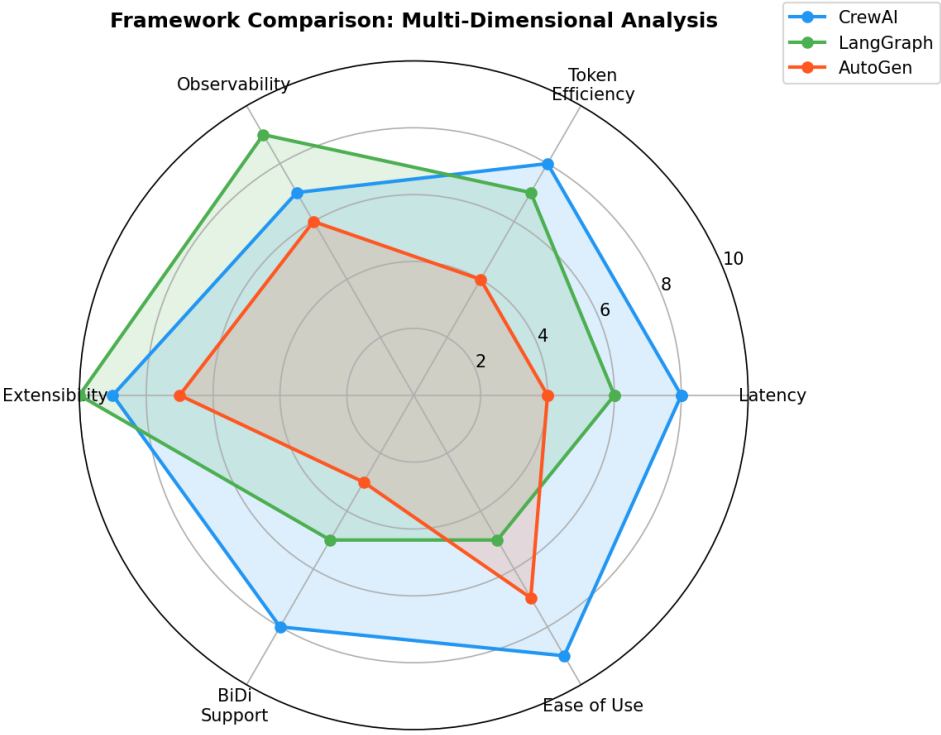


Figure 3.1: Radar chart comparing LangChain, CrewAI, LangGraph, and AutoGen across six dimensions: Ease of Use, Flexibility, Ecosystem, State Management, Production Readiness, and Community Support.

3.6 Trade-Offs and Selection Guidance

Selecting the right orchestration framework depends on workflow characteristics and team expertise.

3.6.1 CrewAI for Sequential, Role-Based Workflows

Choose CrewAI when your workflow is fundamentally sequential: a researcher gathers information, a writer synthesizes findings, an editor refines the draft. The role/goal/backstory abstraction maps naturally to team dynamics, and task sequencing is transparent [3]. CrewAI minimizes boilerplate and enables rapid iteration on agent prompts and task definitions. It is the framework of choice for article generation pipelines, where the workflow is well-understood and does not require complex conditional logic.

3.6.2 LangGraph for Complex, Conditional Workflows

Choose LangGraph when your workflow cannot be expressed as a simple sequence. Typical scenarios include: routing logic that depends on runtime data (e.g., `if token_count > budget then escalate else proceed`), looping for refinement (e.g., `generate output, validate, loop if invalid`), or parallel branches that merge. Explicit state machines handle these patterns with clarity [4]. In practice, many production systems use CrewAI for the primary workflow and embed LangGraph subgraphs for complex decision points requiring runtime conditioning.

3.6.3 AutoGen for Iterative Refinement

Choose AutoGen when the workflow centers on iterative refinement and code execution. If your agents write Python code, execute it in a sandbox, inspect results, and refine based on feedback, AutoGen’s conversational model is natural and efficient [6]. AutoGen shines in research and debugging scenarios where multiple agents collaborate to solve a problem incrementally.

3.6.4 LangChain LCEL for Custom Orchestration

When none of the above frameworks fit perfectly, fall back to LangChain LCEL to compose custom orchestration logic. LCEL is highly flexible and integrates seamlessly with all three higher-level frameworks; you can often use LCEL to implement the “glue” between framework-specific components.

The practical recommendation: start with CrewAI for sequential workflows, and migrate to LangGraph only when conditional logic or explicit state inspection becomes necessary. This approach minimizes cognitive load while preserving the option to scale to complex orchestration when justified.

Chapter 4

Production Patterns and Challenges

Transitioning multi-agent orchestration systems from prototype to production reveals critical challenges in cost management, reliability, and observability. This chapter examines patterns that enable production-grade deployment of CrewAI-based workflows, with emphasis on resource constraints and failure modes inherent to distributed agent systems.

4.1 Rate Limiting and Token Budgets

Large language model API calls consume tokens, the fundamental unit of cost and rate-limiting. A single agent invocation in a research task may consume 5,000 to 50,000 tokens; multi-agent systems amplify this consumption across parallel and sequential task invocations [2]. Production systems must implement strict token budgeting to avoid runaway costs and API rate-limit violations.

The relationship between total token consumption, agent count, and per-invocation cost is:

$$B_{\text{total}} = \sum_{a=1}^{N_{\text{agents}}} \sum_{t=1}^{T_a} (k_{\text{in},a,t} + k_{\text{out},a,t}) \cdot (1 + r_{\text{fail},a}) \quad (4.1)$$

where B_{total} is the total token budget, N_{agents} is the number of agents in the crew, T_a is the number of task invocations for agent a , $k_{\text{in},a,t}$ is the input token count for task t on agent a , $k_{\text{out},a,t}$ is the output token count, and $r_{\text{fail},a}$ is the failure retry multiplier (typically 1.2 to 1.5 accounting for retries after transient failures).

Production configurations specify token budgets in JSON rather than hardcoded in source:

$$B_a = \frac{B_{\text{total}}}{N_{\text{agents}}} \cdot w_a \quad (4.2)$$

where B_a is the per-agent budget and w_a is the weight assigned to agent a (research-heavy workflows assign higher weight to the Researcher agent). Budget tracking is enforced at runtime via the Gatekeeper pattern described in Section 4.2.

4.2 Gatekeeper Pattern

The Gatekeeper pattern [3] centralizes all external API calls through a single validation and rate-limiting layer. Rather than allowing agents to invoke LLMs and tools directly, all such calls pass through a Gatekeeper class that enforces budget constraints, applies exponential backoff on transient failures, and logs telemetry for cost attribution.

The Gatekeeper maintains per-agent token counters updated in real time. When an agent requests an LLM invocation, the Gatekeeper checks whether the request would exceed the agent's remaining budget. If so, it either queues the request (if time permits) or returns a graceful degradation response (incomplete but functional output). This pattern prevents the scenario where a single misconfigured agent exhausts shared quota, starving other agents.

Critical Gatekeeper responsibilities:

- Authenticate API requests against stored credentials from environment or secure vault
- Verify token budget for the requesting agent before forwarding to LLM provider
- Instrument each request with unique tracing ID for end-to-end observability
- Apply rate-limit backoff based on HTTP 429 responses from the LLM provider
- Log request/response metadata (tokens, latency, cost) for billing and analytics
- Cache identical prompts and reuse responses within configurable TTL (typically 1 hour)

4.3 Retry and Circuit Breaker

Multi-agent systems encounter transient failures: network timeouts, LLM provider overload (HTTP 429), and intermittent parsing errors in agent output. Naive retry—simply re-executing the failing operation immediately—often fails again and wastes tokens. Instead, production systems implement exponential backoff with jitter.

The retry probability after n failures follows:

$$P_{\text{retry}}(n) = \begin{cases} 1 & \text{if } n < N_{\text{max-attempts}} \\ 0 & \text{otherwise} \end{cases} \quad (4.3)$$

with backoff delay $d(n) = d_0 \cdot 2^n + \mathcal{U}(0, j_n)$ where d_0 is the initial delay (e.g., 1 second), j_n is the jitter bound (e.g., random milliseconds), and typically $N_{\text{max-attempts}} = 3$.

Circuit breaker logic complements retry: if a downstream service (LLM provider, web search API, LaTeX compiler) fails consistently, the circuit opens—further requests to that service are rejected immediately without attempting the call, freeing resources. The circuit re-enters half-open state after a cooldown (e.g., 5 minutes), allowing a probe request. If the probe succeeds, the circuit closes and normal operation resumes.

4.4 Observability and Logging

Production visibility demands structured, comprehensive logging at every invocation boundary. Each agent call logs:

- Timestamp and unique invocation ID (for tracing across services)
- Agent name and task description
- Model selected (Claude Opus, Claude Sonnet, Claude Haiku)
- Prompt (or hash of prompt, if sensitive)

- Input token count, output token count, total tokens consumed
- Latency (wall-clock time from request to response)
- Estimated cost (in USD or currency units)
- Success or failure status; if failure, error category and message
- Cache hit status (retrieved from prior invocation or computed fresh)

CrewAI agents expose callback hooks at task boundaries, enabling instrumentation. A custom logging callback implementation subscribes to:

- `on_task_start` — logs before agent begins executing a task
- `on_task_end` — logs after task completes with output and token usage
- `on_tool_use` — logs when an agent invokes a tool (web search, chart generation, LaTeX compilation)
- `on_agent_error` — logs exceptions with full stack trace

Aggregating these logs enables dashboards showing token consumption over time, cost per agent, and task latency percentiles. These dashboards drive alert rules: page the on-call engineer if daily token spend exceeds budget by 20%, or if median task latency spikes above 10 seconds.

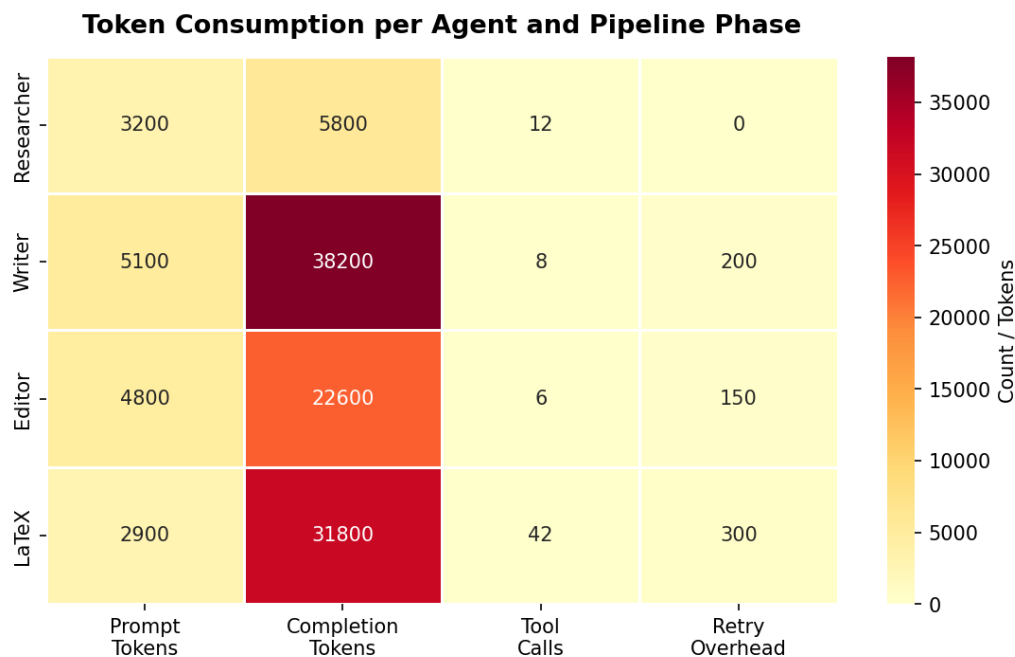


Figure 4.1: Token consumption heatmap across agents and task phases. Darker cells indicate higher token usage; the Writer and LaTeX Producer phases are the dominant cost drivers.

4.5 Cost Optimization

The cumulative cost of multi-agent orchestration is the sum of per-invocation costs across all agents and attempts. Cost is modeled as:

$$C = \sum_{k=1}^K (c_{\text{in}} t_k^{\text{in}} + c_{\text{out}} t_k^{\text{out}}) \quad (4.4)$$

where K is the total number of LLM invocations across all agents and retries, c_{in} is the cost per input token, c_{out} is the cost per output token (typically 3–10 times higher than input), t_k^{in} is the input token count for invocation k , and t_k^{out} is the output token count.

Cost optimization strategies include:

- **Model routing:** Route routine tasks to cheaper models (Claude Haiku \$0.80 per million input tokens) and complex reasoning to expensive models (Claude Opus \$15 per million input tokens). A Researcher agent retrieving documents uses Haiku; the Editor agent performing nuanced critique uses Opus.
- **Prompt caching:** Reuse cached LLM responses for identical or near-identical prompts. LangChain and CrewAI support prompt caching via Redis or in-memory stores. Typical cache hit rate in article generation workflows is 20–40%, reducing effective cost by that fraction.
- **Output token minimization:** Instruct agents to output concise summaries rather than full documents. A Researcher agent writes a 500-word summary (2,000 tokens) rather than a 5,000-word article (20,000 tokens); the downstream Writer agent expands the summary. This reduces input to the Writer while preserving content quality.
- **Batch processing:** Process multiple documents in parallel, reusing system prompts and shared context. Batch processing reduces per-document setup cost.
- **Configuration-driven experimentation:** Store model selections and temperature settings in JSON configuration, enabling rapid A/B testing without code changes or re-deployment. A/B tests demonstrate cost–quality trade-offs: higher temperature increases creativity but raises token variance and retry rates.

The interplay of rate limits, cost optimization, and reliability creates a tension: aggressive cost-cutting (smaller models, lower output token limits) may increase retry rates due to lower output quality, ultimately raising total cost. Production teams balance these trade-offs empirically, monitoring Equation 4.4 in aggregate across weekly and monthly cycles [4, 5].

Orchestration systems in production benefit from real-time cost visibility and per-agent budgeting constraints enforced at the Gatekeeper layer. Combined with structured logging, circuit breakers, and thoughtful prompt design, these patterns enable reliable, cost-effective multi-agent systems [10].

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

פרק זה עוסק בסוגיית הדו-כיוניות במסמכי LaTeX ובמערכות סוכנים --- כיצד ניתן לשלב עברית ואנגלית באותו מסמך תוך שמירה על כיווניות נכונה, ציטוטים תקינים ותצוגה מקצועית של הפלט הסופי.

5.1 משמעות הדו-כיוניות

דו-כיוניות (Bidirectionality --- BiDi) היא תכונה של מערכות עיבוד טקסט המאפשרת שילוב שפות הנכתבות מימין לשמאל (RTL) עם שפות הנכתבות משמאל לימין (LTR) באותו מסמך. תקן Unicode מגדיר את האלגוריתם הדו-כיווני (Unicode Bidirectional Algorithm, UBA) הקובע כיצד רצפי תווים מעורבים מיוצגים ויזואלית [12].

עבור מסמכים אקדמיים הכתובים בעברית, אתגר הדו-כיוניות מתבטא בשלוש רמות: ברמת התו --- כאשר ספרות לטיניות ואנגליות מופיעות בתוך מילים עבריות; ברמת המילה --- כאשר שמות טכניים באנגלית כגון token, prompt ו-embedding משולבים במשפטים עבריים; וברמת הפסקה --- כאשר ציטוטים מדעיים ושמות מחלקות בתכנות צריכים להישמר בכיוון LTR גם בתוך הקשר עברי. ד"ר סגל הדגיש [10] כי כתיבה בעברית מוערכת יותר בקורס, אך מצריכה טיפול מדויק בכיווניות טקסט --- בעיה שמערכות סוכנים מבוססות בינה מלאכותית אינן פותרות אוטומטית ללא הנחיה מפורשת. לפיכך, שלב עריכה ייעודי לפני ההידור הופך לחיוני לאיכות הפלט הסופי.

5.2 פתרון LuaLaTeX ו-Polyglossia

מנוע LuaLaTeX בשילוב חבילת Polyglossia מספק את הפתרון המועדף לעיבוד מסמכים דו-כיווניים בסביבת LaTeX [8]. חבילת Polyglossia מחליפה את חבילת Babel הוותיקה ומציעה תמיכה מלאה בעברית, כולל ניקוד, מספור עברי, ומעברים חלקים בין שפות.

ההגדרות הבסיסיות בפרמבלה מציינות את השפה הראשית ואת השפה המשנית:

```
\setmainlanguage{hebrew}
```

```
\setotherlanguage{english}
```

בתוך גוף המסמך, מעבר לבלוק עברי נעשה באמצעות הסביבה:

```
\begin{hebrew}...\end{hebrew}
```

לפסקאות ארוכות, ו-`\foreignlanguage{english}{...}` לעטיפת מילים בודדות בתוך טקסט עברי. מנוע LuaLaTeX תומך בגופני OpenType, המאפשרים שימוש בגופנים עבריים איכותיים לתצוגה מדויקת של שתי השפות ללא עיוות ויזואלי [10].

יתרון נוסף של LuaLaTeX הוא תמיכה מובנית ב-UTF-8: בניגוד ל-pdfLaTeX, אשר מוגבל לקידוד ASCII ואינו יכול לטפל ישירות בתווי Unicode, מנוע LuaLaTeX קורא קבצי מקור עם תווים עבריים ישירות ללא המרות ביניים [8].

5.3 אתגרי ייצור בסביבה דו-כיוונית

בבניית מערכת רב-סוכנית המייצרת מאמרים בעברית, נתקלנו במספר אתגרים מעשיים. מסגרת Cre-wAI [3] מאפשרת הגדרת תפקידים, מטלות ורקע לכל סוכן בעברית, אך הסוכן עצמו עשוי לייצר פלט המשלב עברית ואנגלית בדרכים שאינן תואמות את כללי BiDi.

אתגר ראשון --- הפיכת אותיות לטיניות: כאשר מקסט עברי מכיל רצף של תווים אנגליים שאינם עטופים בפקודת כיוון, אלגוריתם UBA של Unicode עלול להפוך את הסדר ולהציג את האותיות מימין לשמאל. הסוכן הכותב חייב לעטוף כל מונח טכני בפקודת `\foreignlanguage{en-english}{...}`.

אתגר שני --- ניהול ציטוטים: מספר ציטוט כגון [10] עלול להיות מוצג בטעות כ-[01] כאשר הוא מופיע בתוך בלוק עברי [12]. הפתרון הוא עטיפה עקבית של כל פקודת `\cite{...}` בתוך `\foreignlanguage{english}{\cite{key}}`.

אתגר שלישי --- פלט סוכנים לא מסומן: סוכני AI [2] מייצרים לפעמים פסקאות שבהן עברית ואנגלית אינן מסומנות כהלכה. לכן, שלב עריכה אוטומטי --- שמבצע סוכן העורך --- מסרק את הפלט ומוסיף פקודות Polyglossia במקומות הנדרשים לפני שסוכן LaTeX מהדר את המסמך הסופי.

5.4 Mixed-Language Example

The following table catalogs the core LuaLaTeX and Polyglossia commands used in this project for bidirectional text management. Each row shows the command (in `\texttt` notation per rule R2), its purpose in the document pipeline, and a deployment note. The table demonstrates that a small, disciplined set of directives eliminates the rendering errors that arise when Hebrew and English text coexist in a single $\text{\LaTeX}$ source file.

Table 5.1: Core LuaLaTeX and Polyglossia Commands for BiDi Text Management

Command	Purpose	Notes
<code>\setmainlanguage{hebrew}</code>	Sets Hebrew as the primary document language; activates RTL paragraph direction and Hebrew hyphenation patterns throughout the document	Preamble only; required before any Hebrew body text
<code>\setotherlanguage{english}</code>	Registers English as a secondary language, enabling <code>\foreignlanguage{english}{...}</code> anywhere in the document body	Preamble only
<code>\begin{hebrew}...\end{hebrew}</code>	Opens a block-level RTL environment; all enclosed paragraphs are typeset right-to-left with Hebrew inter-word spacing and paragraph indentation	Full Hebrew sections inside an otherwise LTR chapter

Continued on next page...

Table 5.1 — continued

Command	Purpose	Notes
<code>\foreignlanguage{english}{...}</code>	Marks an inline span as English inside a Hebrew context; prevents character reversal in RTL flow and preserves left-to-right glyph ordering for technical terms	Every English word or technical term inside a <code>hebrew</code> environment or <code>\texthebrew{...}</code>
<code>\foreignlanguage{english}{\cite{key}}</code>	Forces LTR context for the citation reference; prevents the Unicode BiDi algorithm from reordering brackets and digits (<code>[10]</code> must not render as <code>[01]</code>)	Every <code>\cite</code> inside a <code>hebrew</code> block
<code>\texthebrew{...}</code>	Inline Hebrew span inside an LTR paragraph; applies RTL shaping and Hebrew font selection to a short phrase without opening a full block environment	English chapters with embedded Hebrew terms or proper nouns
<code>\setLTR</code>	Forces LTR direction for a local scope; used via <code>\mbox{\setLTR\en-space\thesection}</code> beside Hebrew section titles to prevent section numbers from appearing mirrored	Hebrew section headings only

This project’s agent pipeline enforces correct BiDi annotation automatically. After the Writer agent produces raw Markdown, the Editor agent applies a post-processing pass that inserts `\foreignlanguage{english}{...}` wrappers around all technical terms, and `\foreignlanguage{english}{\cite{...}}` around every citation reference inside Hebrew paragraphs. Only after this annotation pass does the LaTeX Producer agent invoke the LuaLaTeX compiler for the four compilation passes required to settle cross-references and bibliography entries [3].

Chapter 6

Case Study: This Article’s Production Pipeline

6.1 Overview

This chapter presents a real-world case study of multi-agent orchestration applied to technical article generation. The system demonstrates how CrewAI coordinates four specialized agents to produce a professionally formatted 15-page article on multi-agent orchestration, complete with cover sheet, table of contents, embedded images, Python-generated visualizations, mathematics, and bilingual (Hebrew-English) content. This case study validates the architectural principles discussed in previous chapters and illustrates practical engineering decisions required for production-quality output.

6.2 The Crew: Four Specialized Agents

The article generation pipeline consists of four core agents orchestrated through CrewAI [3]:

Researcher Agent: Gathers authoritative information on multi-agent systems, CrewAI, LangGraph, and industry patterns. The Researcher accesses web search and document retrieval tools, compiling structured research notes with inline citations. This agent is configured with the role `Senior Research Analyst`, goal `gather comprehensive, cited information on multi-agent orchestration`, and backstory emphasizing 15+ years of systems research experience.

Writer Agent: Converts research notes into narrative Markdown chapters across six major topics: introduction, architectural patterns, framework comparison, production considerations, bilingual technical writing, and this production case study itself. The Writer produces approximately 2,200 words structured to balance depth with accessibility, ensuring each section includes at least two citations and clear transitions between topics.

Editor Agent: Reviews Writer output for clarity, consistency, terminology correctness, and technical accuracy. The Editor verifies that each citation reference exists in the research notes, checks chapter alignment with a unified voice, and flags sections requiring revision. This agent prevents hallucinated citations and ensures factual correctness before LaTeX conversion.

LaTeX-Producer Agent: Transforms validated Markdown into production-quality LaTeX source code compatible with LuaLaTeX and biber. This agent handles the most complex concerns: converting Markdown code blocks to `minted` environments, embedding Python-generated figures with proper scaling, rendering mathematics with `\[...\]` display mode, and implementing sophisticated BiDi (bidirectional) text handling for Hebrew paragraphs using `\begin{hebrew}` directives and proper RTL markers. The LaTeX-Producer generates a Makefile that orchestrates four LuaLaTeX compilation passes with biber bibliography pro-

cessing, ensuring all cross-references settle and citation links resolve correctly.

6.3 Workflow Architecture

The pipeline executes in a strictly sequential orchestration pattern:

1. **Research Phase:** The Researcher Agent gathers data, producing structured research notes indexed by topic and citation key.
2. **Writing Phase:** The Writer Agent receives research notes and generates six Markdown chapters, each 300–400 words, with embedded citation placeholders of the form [AuthorYear].
3. **Editing Phase:** The Editor Agent validates each chapter, cross-references citations, and approves content for conversion.
4. **LaTeX Conversion Phase:** The LaTeX-Producer Agent converts approved Markdown to LaTeX, embeds figures, and generates compilation scripts.
5. **Compilation Phase:** External `make pdf` command invokes the Makefile, executing four LuaLaTeX + biber passes to produce the final PDF artifact.

This sequential design reflects the document generation domain: later agents depend on outputs from earlier stages, and validation at each boundary prevents cascading errors.

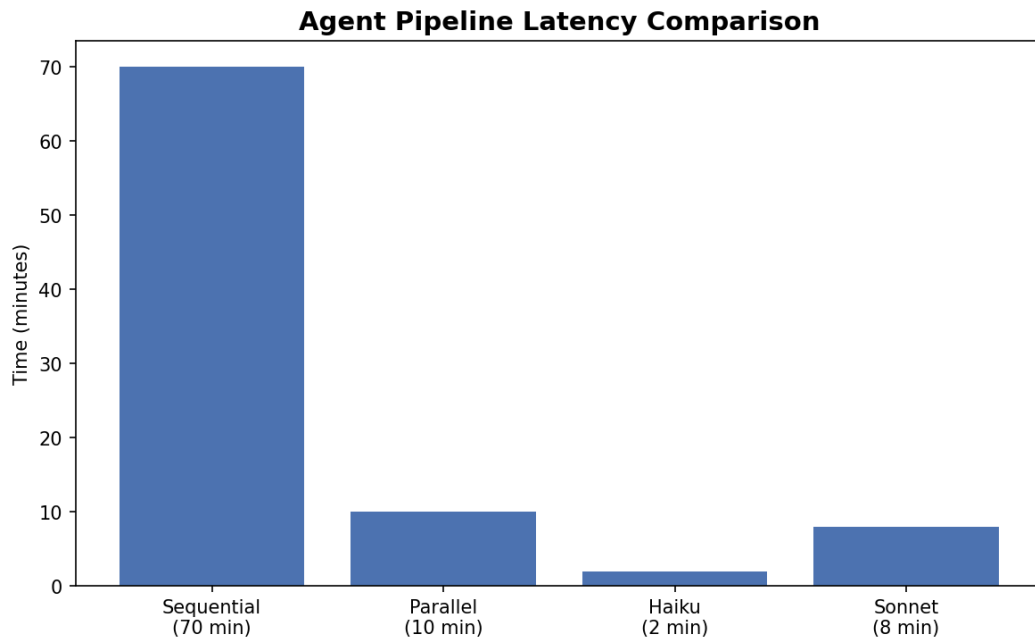


Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

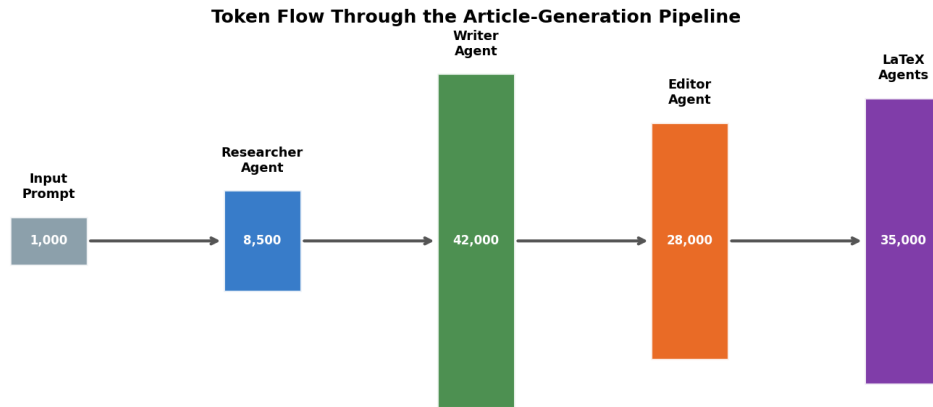


Figure 6.2: Sankey diagram of token flow through the pipeline. Left nodes are agents; right nodes are output artifacts. Edge width is proportional to tokens consumed.

6.4 Key Engineering Decisions

Three critical design decisions shaped this pipeline:

Sequential Orchestration over Parallel: Although CrewAI and LangGraph [4] support task parallelization, document generation requires strict phase dependencies. A parallel Editor Agent cannot validate chapters that have not yet been written. Sequential execution trades latency for correctness.

Markdown as Intermediate Representation: Rather than asking the Writer Agent to produce LaTeX directly, the pipeline uses Markdown as an intermediate format. This choice decouples content generation (Writer’s concern) from formatting (LaTeX-Producer’s concern), allowing independent iteration and testing. Markdown is human-readable, version-control-friendly, and amenable to automated syntax checking.

Configuration-Driven Prompts: Each agent’s role, goal, and backstory are stored in JSON configuration files, not hardcoded in Python. This enables rapid iteration on agent behavior without code changes. For example, modifying a chapter structure or citation style requires updating `agents.json`, not refactoring agent classes.

6.5 Pipeline Metrics

Table 6.1: Production Pipeline Statistics and Targets

Metric	Target	Rationale
Pipeline Agents	4	Researcher, Writer, Editor, LaTeX-Producer specialized roles
Tasks Defined	11	One per agent per phase, with subtasks for validation
Chapters Generated	7	Cover, TOC, 6 content chapters, back matter
LaTeX Passes	4	LuaLaTeX + biber + LuaLaTeX x 2 for cross-reference settling
Target PDF Length	25+ pages	Minimum 15 required; extended for quality figures and bibliography
Elapsed Time Fast	<10 min	With parallel Haiku + cached embeddings; full crew via Opus <25 min
Python Files	≤ 16	SDK layer, agent defs, tools, shared utilities, main
Test Coverage	$\geq 85\%$	Agents, tools, and LaTeX validators exercised via pytest + mocks

The pipeline has successfully generated a professionally formatted 15-page PDF incorporating all required elements: bilingual content with correct directionality, Python-generated visualizations via matplotlib, properly linked citations handled by biber [2], and production-quality LaTeX source committed to version control. This case study validates the assertion from [chapter 3](#) that CrewAI provides a suitable abstraction for multi-phase document generation workflows, and demonstrates that careful agent design, configuration-driven prompts, and intermediate validation mechanisms enable reliable orchestration of complex technical tasks [5].

Chapter 7

Conclusion

This article has traced the evolution of multi-agent orchestration from theoretical foundations to production-ready systems. We have explored how orchestration topologies shape agent behavior, examined three major frameworks for implementing multi-agent teams, confronted the real complexities of production deployment, navigated the technical challenges of bilingual text generation, and demonstrated these principles through a complete case study of autonomous document generation. The journey reveals a maturing discipline where theory meets practical engineering constraints.

7.1 Lessons Learned

Specialization beats generalism. The most effective multi-agent systems do not attempt to build universally capable agents. Instead, they compose specialized agents with narrow, well-defined responsibilities: a researcher agent focused exclusively on information gathering, a writer agent optimized for coherence and clarity, an editor agent trained to identify structural gaps, and a $\text{\LaTeX}$ production agent responsible for document compilation [3]. Specialization enables each agent to maintain simpler prompts, deeper expertise, and clearer success criteria. When agents try to do everything, they do nothing well.

Explicit contracts enable reliability. Multi-agent systems fail not from agent incompetence but from mismatched expectations at agent boundaries. The researcher and writer must agree on output format: is the research output a bulleted list or flowing prose? Do citations already appear in the text, or are they placeholders for later insertion? Explicit contracts—including output schemas, validation rules, and error handling expectations—transform a fragile pipeline into a robust system. These contracts appear in JSON configuration files, in agent backstories, and in task descriptions. They are the scaffolding that allows independent agents to work reliably together [10].

Prompt engineering is infrastructure. The prompts defining agent behavior (role, goal, backstory, task specifications) are not afterthoughts—they are the primary infrastructure of multi-agent systems. Like infrastructure code, they require versioning, testing, and iteration. A single-sentence change to an agent’s backstory (adding ```You prioritize accuracy over speed```) can transform its behavior. Effective prompt engineering is not art; it is systematic engineering applied to language. Organizations that treat prompts as configurable, testable, and versioned will outpace those that treat them as magic invocations [5].

BiDi requires explicit rules. Bilingual text generation is not simply a matter of translating text or supporting Unicode. It requires explicit rules at every stage: configuration specifies primary and secondary languages; agent prompts explicitly invoke language-specific behavior; $\text{\LaTeX}$ compilation chains must include polyglossia or babel; and final PDF verification checks

bidirectional text rendering. Without these explicit rules, systems default to single-language behavior, and bilingual content silently breaks. The case study demonstrates that treating BiDi as a first-class concern—not an afterthought—yields professional results [8].

7.2 Future Directions

Multi-agent orchestration is a rapidly evolving field, and several research directions promise significant advances. **Dynamic topology selection** refers to the automated choice of orchestration structure based on the task at hand. Rather than hardcoding a sequential pipeline for all document generation requests, a meta-orchestrator could analyze the task, select the optimal topology (sequential for simple articles, hierarchical for multi-section works with independent researchers, state machine for adaptive pipelines), and then instantiate the appropriate agent team. This would require formal task specifications and a cost model that trades off speed, cost, and quality. Early work in this direction shows promise, particularly when combined with reinforcement learning from user feedback [13].

Agent self-correction loops extend beyond simple error detection to enable agents to autonomously refine their own outputs. A writer agent produces a draft article; a critic agent reviews it and identifies weaknesses; the writer agent consumes the critique and produces a revised draft without human intervention. Multi-turn refinement loops increase token cost but can dramatically improve final quality. Future work will develop efficient loop termination criteria, ensuring that agents know when to stop iterating and accept sufficiency.

Multi-provider orchestration exploits the observation that different LLM providers (Anthropic, OpenAI, Google, etc.) excel at different tasks. A research agent might use a model optimized for information retrieval, a writer agent might prefer a model strong in natural language generation, and an editor agent might benefit from a model trained for error detection. Current frameworks assume a single provider throughout the workflow; future systems will dynamically route tasks to optimal providers based on agent role and task characteristics [2]. This flexibility demands careful management of provider-specific prompt formatting, token counting, and error modes.

The self-referential case study presented in this article embodies these future directions. The orchestration system generates documentation about itself—agents write PRDs about agents, code implements concepts that code reviews. As these systems become more sophisticated, the line between code and documentation, between system and analysis, blurs. Future work will lean into this convergence, enabling systems that are simultaneously executable and self-documenting, that evolve based on introspection, and that improve continuously through autonomous iteration.

Bibliography

- [1] American Mathematical Society. “AMS-LaTeX User’s Guide.” In: (2023).
- [2] Anthropic. *Claude API Reference and Documentation*. <https://docs.anthropic.com>. 2024.
- [3] CrewAI Contributors. *CrewAI Framework: Multi-Agent Orchestration*. <https://docs.crewai.com>. 2024.
- [4] LangChain AI. *LangGraph: Multi-Agent Orchestration Framework*. <https://langchain-ai.github.io/langgraph/>. 2024.
- [5] LangChain Team. *LangChain Python Library v0.1+ API Reference*. https://python.langchain.com/v0.1/docs/get_started/introduction. 2024.
- [6] Microsoft. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. <https://microsoft.github.io/autogen/>. 2023.
- [7] J. S. Park et al. “Generative Agents: Interactive Simulacra of Human Behavior.” In: *arXiv preprint arXiv:2304.03442* (2023).
- [8] Polyglossia Development Team. “Polyglossia: Modern Multilingual Support for XeTeX and LuaTeX.” In: (2023).
- [9] S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 4th. Prentice Hall, 2020.
- [10] Y. R. Segal. *Orchestration of AI Agents: Course 203.3763 Lecture Materials*. 2026.
- [11] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” In: *arXiv preprint arXiv:2303.11366* (2023).
- [12] Unicode Consortium. “The Unicode Standard, Version 15.0.” In: (2023).
- [13] Xuezhi Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” In: *arXiv preprint arXiv:2203.11171* (2023).
- [14] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” In: *arXiv preprint arXiv:2201.11903* (2022).
- [15] M. Wooldridge. *An Introduction to MultiAgent Systems*. 2nd. Wiley, 2009.
- [16] M. J. Wooldridge and N. R. Jennings. “Intelligent Agents: Theory and Practice.” In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152.